Date: 07 October 2026

KEYCLOCK: PROJECT REPORT

Time-Based OTP Authentication and Member Messaging

Keyclock is a web application that combines password-based registration with a six-digit authenticator code. After successful verification, members can access a dashboard, view registered users, and send messages to other members.

Objective:

The objective of this project is to demonstrate how a Python backend and a Next.js frontend can implement time-based one-time password (TOTP) authentication. A password proves knowledge of a credential; the authenticator code demonstrates access to the enrolled secret.

Main Features:

- Registration with a name, email address, and password; duplicate emails are checked without regard to letter case.
- QR-code enrollment compatible with Google Authenticator and other standard TOTP applications.
- Password and OTP login, eight one-use recovery codes, and server-revocable sessions.
- A dashboard with Inbox, Sent, message counts, and a registered-users directory.
- Show/Hide password controls, form validation, and clear failure messages.

Technologies Used:

Component	Technology and purpose
Frontend	Next.js, React, TypeScript, and CSS; hosted on Vercel.
Backend	Python, FastAPI, and Uvicorn; hosted on Render.
Database	SQLAlchemy with PostgreSQL in deployment; SQLite for local development.
Authentication	PyOTP, qrcode, Argon2 password hashing, and Fernet encryption.

This report describes the implemented project. Supabase is a compatible PostgreSQL hosting option; migration to it is not assumed to have been completed.

1. SYSTEM ARCHITECTURE

How the Components Work Together:

The browser communicates with Next.js on the same website origin. Next.js route handlers forward approved requests to FastAPI. The Python service verifies credentials, checks permissions, and reads or writes database records.

Layer	Responsibility
Browser	Displays forms, dashboard, messages, and member information. Sends requests to /api/auth/... on the frontend.
Next.js on Vercel	Checks the request origin, forwards JSON to Render, and keeps the session token in an HTTP-only cookie.
FastAPI on Render	Performs registration, TOTP verification, rate limiting, session validation, and messaging operations.
PostgreSQL	Persists users, session tickets, recovery-code digests, throttles, and messages.
Authenticator app	Stores the enrolled secret and computes changing codes locally using the current time.

Request Flow:

1. The user submits a form in the browser.

2. The frontend calls its own Next.js API route. Mutating requests must use JSON and match APP_ORIGIN.

3. Next.js adds X-Internal-Key and forwards the session token as a Bearer authorization header.

4. FastAPI validates the server key and, for protected actions, the ticket scope and expiry.

5. FastAPI returns a result. Next.js removes any returned token from JSON and stores it in the session cookie.

Why Use a Frontend API Layer?

This design keeps the internal API key out of browser JavaScript and avoids cross-site cookie handling between Vercel and Render. The production cookie is Secure, HTTP-only, and SameSite=Lax. Authentication responses are marked no-store.

2. REGISTRATION AND AUTHENTICATOR SETUP

Registration Procedure:

1. The user enters a name (1-100 characters), valid email, and password (12-128 characters). Leading and trailing spaces are removed from the name.

2. The backend normalizes the email to lowercase and checks for an existing account. A database unique constraint also prevents duplicate registration during concurrent requests.

3. The password is hashed with Argon2. A random Base32 authenticator secret is generated and stored encrypted with Fernet.

4. The backend issues a limited enrollment ticket lasting 10 minutes. This ticket cannot open the dashboard.

QR-Code Enrollment:

The enrollment endpoint decrypts the user's secret and creates an otpauth provisioning URI containing the account email and issuer, Keyclock. The URI is encoded as a PNG QR code. The user scans it in an authenticator or enters the displayed setup key manually.

The user must enter a valid code from the app to confirm that enrollment worked. Confirmation enables TOTP, invalidates earlier tickets, generates eight recovery codes, and creates a one-hour authenticated session. Recovery codes are displayed once and should be saved privately.

How a Six-Digit Code Is Generated:

The phone and server share the same secret. They independently calculate a value from that secret and the current 30-second time step. PyOTP uses HMAC-SHA-1 and converts the result to six decimal digits, preserving leading zeros.

Concept	Value in Keyclock
Time counter	T = floor(Unix time / 30)
Code calculation	TOTP = HOTP(shared secret, T), formatted as six digits
Accepted time steps	Current step, one previous step, or one next step
Replay protection	A code is accepted only if its time counter is greater than the stored last_counter.

Codes are generated by the authenticator without SMS or email delivery. Phone and server clocks should be synchronized. The QR code and manual setup key contain the secret and must be kept private.

3. LOGIN, RECOVERY, AND SESSION CONTROL

Normal Login:

1. The user submits an email and password. FastAPI verifies the stored Argon2 hash; incorrect credentials return an error.

2. For an enrolled account, a correct password creates a five-minute MFA challenge. Password verification alone does not grant dashboard access.

3. The user enters the current six-digit authenticator code. The backend checks the allowed time steps and rejects a previously accepted step.

4. Successful verification consumes the challenge and creates a one-hour session. The dashboard then loads the account and messages.

If an account has not completed enrollment, login returns to QR setup instead. The Show/Hide password control changes only the input display; passwords default to hidden when switching screens.

Recovery Codes:

After password verification, a user who cannot access the authenticator may select the recovery-code option. Each code is a random 16-character hexadecimal value. Only its SHA-256 digest is stored. A successful use deletes that digest, so the same code cannot be reused.

A recovery code restores a sign-in session; it does not register a replacement phone. Authenticator replacement and recovery-code regeneration are not implemented in the current version.

Ticket Types:

Scope	Lifetime	Allowed purpose
enroll	10 minutes	View the setup secret and confirm enrollment.
mfa	5 minutes	Complete password-verified OTP or recovery verification.
session	1 hour	Access the profile, member directory, and messages.

Logout and Expiry:

Tickets are random opaque values, not JWTs. The database stores their SHA-256 digests, user IDs, scopes, and expiry times. Logout deletes the current ticket and clears the browser cookie. Expired or incorrectly scoped tickets are rejected.

4. DASHBOARD AND DATABASE DESIGN

Member Dashboard:

After authentication, the dashboard shows account details and received/sent message totals. Inbox and Sent display the latest 50 messages per folder. Refresh loads new data; this version does not use real-time delivery or push notifications.

The Registered users section shows names and emails to fully signed-in members. The current account is marked You, and incomplete accounts show Setup pending. Results load in batches of 50. Message and Reply buttons fill the recipient field without automatically sending anything.

Sending a Message:

The sender enters another member's email and a message of 1-2,000 characters. The backend derives the sender from the authenticated ticket, checks that the recipient has completed setup, and rejects self-messages. Saved messages appear in the sender's Sent folder and the recipient's Inbox.

Database Tables:

Table	Stored information	Purpose
users	ID, name, email, password hash, encrypted secret, enabled flag, last counter	Account identity and authenticator state.
tickets	Token digest, user ID, scope, expiry	Enrollment, login challenges, and sessions.
recovery_codes	Code digest and user ID	Single-use recovery verification.
throttles	Hashed bucket key, count, expiry	Shared fixed-window attempt limits.
messages	ID, sender ID, recipient ID, body, timestamp	Persistent member-to-member messages.

Relationships and Access:

A user may have several tickets, recovery codes, and sent or received messages. Foreign keys connect these records to users. Mailbox queries filter by the authenticated user ID, so a member cannot request another member's inbox through the API.

Message bodies are stored as plain text in the database; they are not end-to-end encrypted. Registration checks email uniqueness but does not verify ownership of the email address.

5. API ROUTES AND SECURITY CHECKS

The browser uses /api/auth/... on Vercel; the corresponding Python endpoints use /auth/.... Each /auth route requires the internal server key. Additional ticket requirements depend on the action.

Method and backend path	Purpose / required ticket
POST /auth/signup	Create an account; returns an enrollment ticket.
POST /auth/login	Verify the password; returns an MFA or enrollment ticket.
GET /auth/enroll	Return the QR code and secret; enrollment ticket.
POST /auth/enroll/confirm	Verify setup and return recovery codes; enrollment ticket.
POST /auth/verify	Verify OTP or recovery code; MFA ticket.
GET /auth/me	Return the account name, email, and status; session ticket.
GET /auth/users	List registered users with a pagination cursor; session ticket.
GET /auth/messages	Return the member's Inbox and Sent; session ticket.
POST /auth/messages	Send a message to another active account; session ticket.
POST /auth/logout	Revoke the supplied ticket.
GET /health	Public database connectivity check; returns status: ok.

Attempt Limits:

- OTP or recovery verification: 5 attempts per account per 5-minute window.
- Password login: 10 attempts per email and 300 globally per 5-minute window.
- Registration: 3 attempts per email and 50 globally per hour.
- Sending messages: 20 attempts per user per 5-minute window.

Limits include successful requests and use fixed time windows. Database updates make OTP acceptance and challenge/recovery-code consumption transactional. Input validation, no-store responses, exact-origin checks, and token scoping provide additional protection.

6. DEPLOYMENT AND CONFIGURATION

Hosting Arrangement:

The Next.js frontend is deployed from the frontend directory to Vercel. The Python service is deployed from backend to Render. PostgreSQL stores persistent data; the local SQLite file is only for development.

Service	Build and start configuration
Vercel	Framework: Next.js; root: frontend. Install: npm ci; build: npm run build.
Render	Root: backend; build: pip install -r requirements.txt. Start: uvicorn app:app --host 0.0.0.0 --port $PORT. Health-check path: /health.

Required Environment Variables:

Location	Variable	Meaning
Render	DATABASE_URL	PostgreSQL connection string.
Render	TOTP_ENCRYPTION_KEY	Fernet key used to encrypt authenticator secrets.
Render	TOTP_ISSUER	Display name in the authenticator, e.g. Keyclock.
Both	INTERNAL_API_KEY	Matching private key for server-to-server requests.
Vercel	BACKEND_URL	Render backend HTTPS origin.
Vercel	APP_ORIGIN	Exact frontend HTTPS origin, without a trailing slash.

Do not place these secrets in frontend JavaScript or commit them to Git. A missing frontend variable produces Server configuration is incomplete. A request to the backend root may return 404 because no homepage route is defined; use /health to check it.

Database Hosting and Availability:

Supabase can replace Render PostgreSQL through DATABASE_URL. Existing accounts and messages require a database migration, and the same encryption key must be retained for existing authenticator secrets. This is a hosting option, not a confirmed migration in this report.

An external scheduler can periodically request /health to reduce idle sleep. No scheduler configuration is included in the current frontend code, and scheduled requests do not guarantee service availability.

7. TESTING, LIMITATIONS, AND CONCLUSION

Verification Performed:

The backend test suite was rerun for this report on 07 October 2026: 22 tests passed. The tests use isolated in-memory SQLite databases. During development, the Next.js production build and the local proxy integration check also passed.

Area	Behavior verified
Registration	Required and trimmed names; case-insensitive duplicate emails; existing-database name migration.
Authentication	Enrollment, successful login, wrong ticket scope, expiry, OTP replay rejection, leading-zero codes, and logout.
Recovery and limits	One-use recovery codes, password/OTP attempt limits, and the internal API key requirement.
Messaging	Delivery and replies, mailbox isolation, invalid recipients, content validation, limits, and recent-message ordering.
Member directory	Signed-in access, restricted fields, self/pending status, and pagination without overlapping records.

These checks do not constitute a production security audit. PostgreSQL concurrency, live deployment configuration, and external scheduler operation still require separate environment-specific verification.

Current Limitations and Future Work:

- Add email ownership verification, password reset, authenticator replacement, and recovery-code regeneration.
- Add message blocking/reporting, optional notifications, and access to older message history.
- Add database migrations, backup/restore procedures, and controls for managing sessions across devices.
- Consider passkeys/WebAuthn for phishing-resistant authentication. TOTP can still be stolen through phishing.

Conclusion:

Keyclock combines password-plus-authenticator login, scoped sessions, and member messaging. Its design separates the browser interface, Python authentication logic, and database storage to support a practical web application.

Implementation References:

Backend: app.py, test_auth.py, and smoke_proxy.py.
Frontend: page.tsx, dashboard.tsx, members.tsx, and the auth API route handler.
Deployment: render.yaml and the project environment-variable examples.